No Threshold Is Safe: Oracle Bounds on Similarity-Based Caching for Text-to-SQL

Ramakrishna Bobba
https://github.com/rbobba/semantic-cache-inversion

Abstract

Semantic caching for text-to-SQL reuses a stored query when a new question is similar enough in embedding space. We show that this approach has a hard ceiling: even an *oracle* per-entry threshold—one that picks the best possible threshold for every cached entry individually—recovers at most 6–17% of reusable queries at zero false reuse, across 15 embedding models from nine publishers (2021–2026). The information needed to separate safe reuse from unsafe reuse is not in the similarity score.

A structured guard that matches on an extracted signature—operation, measure, grouping and exclusion—achieves 77–100% correct reuse on 480 human-written questions across four analytics domains, with two extractor vendors. This guard design is prior art [3, 8]; our contribution is testing it on held-out human wording with pre-registered decision rules.

We also document an extraction hazard not previously reported: excluded values ("everything except travel") are silently dropped 4–40% of the time by a small extraction model (4% in healthcare, 35–40% in the other three domains), while the same value is extracted correctly when included. The residual risk is date semantics: rolling periods and explicit date ranges defeat every design tested.

Dataset, code and all model outputs: https://github.com/rbobba/semantic-cache-inversion

1 Introduction

Semantic caching reduces latency and cost in text-to-SQL systems by reusing a stored SQL query when a new natural-language question is close enough in embedding space [1, 11]. The key decision is the similarity threshold: too low and the cache misses; too high and it serves wrong SQL silently.

The practitioner's assumption is that better threshold tuning, or a learned per-entry threshold [9, 12], will make the cache safe. Several recent systems already go further: COGNI gates reuse on entity overlap, intent and polarity checks [8]; Bindschaedler matches on exact OLAP intent signatures [3]; AWS validates extracted parameter formats against template placeholders [4]. But the failure mode these systems address has been asserted, not measured: COGNI states that period-differing queries "often exceed 0.95 cosine similarity" without running an embedding baseline [8], and Bindschaedler argues that similarity caches "cannot distinguish" total from average revenue without quantifying it [3].

This paper contributes three results:

1. **An oracle bound.** We compute the ceiling for *any* threshold-based similarity method: even with a perfect per-entry threshold, correct reuse at zero false reuse is at most 6–17% (Section 3). This closes the threshold-tuning direction, including learned methods such as vCache [9].
2. **An extraction hazard.** Excluded values are dropped by the extractor 4–40% of the time, while the same value is extracted correctly when included (Section 4). This asymmetry has not appeared in prior error taxonomies.

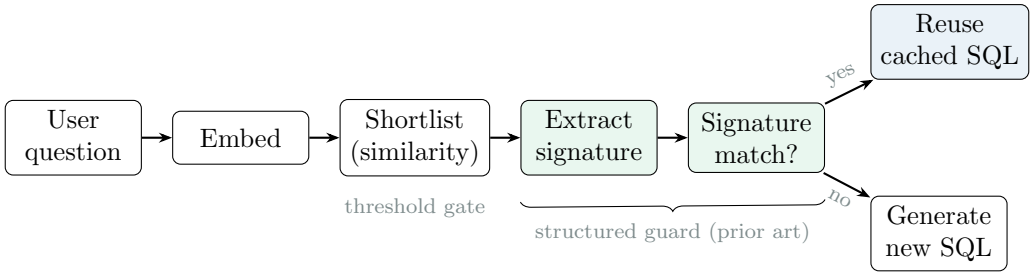


Figure 1: A semantic cache pipeline with a structured guard. The similarity shortlist is a necessary but insufficient pre-filter; the guard checks that the extracted parameters match the cached entry's signature. This design is described by Rathore et al. [8] and Bindschaedler [3]; we measure its performance on held-out human wording.

3. **Guard tested on held-out human wording.** We evaluate the structured-guard design of Rathore et al. [8] and Bindschaedler [3] on 480 questions written by people who never saw the templates, with two extractor vendors and pre-registered decision rules (Section 5).

We do not propose a new caching method. The guard design is prior art. Our contribution is the measurement and the honest reporting of where it breaks.

2 Setup

Task. A text-to-SQL semantic cache stores (question, SQL) pairs. When a new question arrives, the cache embeds it, finds the most similar stored question, and—if similarity exceeds a threshold—returns the stored SQL without calling the language model. A *correct* reuse returns the same SQL intent the new question needs; a *false* reuse returns SQL for a different intent.

Models. We test 15 embedding models from nine publishers: sentence-transformers (MiniLM, mpnet), BAAI (bge), intfloat (e5), Mixedbread (mxbai), Snowflake (arctic), Qwen (Qwen3-0.6B), Google (text-embedding-004, gemini-embedding-001), OpenAI (text-embedding-3-small, 3-large), and Voyage (voyage-3-large, 3.5, 4). Release years span 2021–2026.

Domains. Four analytics domains: personal finance, retail, healthcare operations and IT support. Each domain has 125–145 cached questions generated from domain-specific templates with controlled parameter variations (period, entity, measure and exclusion).

Human questions. 480 questions written by three volunteers who never saw the templates, across all four domains (120 per domain). Each question targets a specific cached template but uses the writer's own wording.

Extractors. Two LLM-based extractors: Google Gemini (gemini-3.1-flash-lite) and OpenAI (gpt-6-luna), each run three times for stability.

Pre-registration. Every hypothesis, decision rule and prediction was committed before its run. Three stages recorded SHA-256 hashes of their pre-registration text. Predictions that turned out wrong are kept in place, not edited. The full record is in the repository's `STUDY_LOG.md`.

Table 1: Correct reuse at zero false reuse (%). The oracle per-entry threshold is the ceiling for any threshold-based similarity method. The template cache with oracle parameters shows what extracted structure recovers. Four representative models are shown; the full table (15 models) is in the appendix.

Model (year)	Global thresh.	Oracle per-entry	Template (oracle params)	Possible
MiniLM-L6-v2 (2021)	0.8	7.1	97.3	53.2
bge-base-en-v1.5 (2023)	0.5	8.1	97.3	53.2
text-emb-3-large (2024)	1.2	9.8	97.3	53.2
gemini-emb-001 (2025)	0.2	10.2	100.0	53.2
voyage-4 (2026)	6.3	16.8	97.3	53.2
Range (all 15)	*0.2–6.3*	*6.4–16.8*	*96.2–100.0*	*53.2*

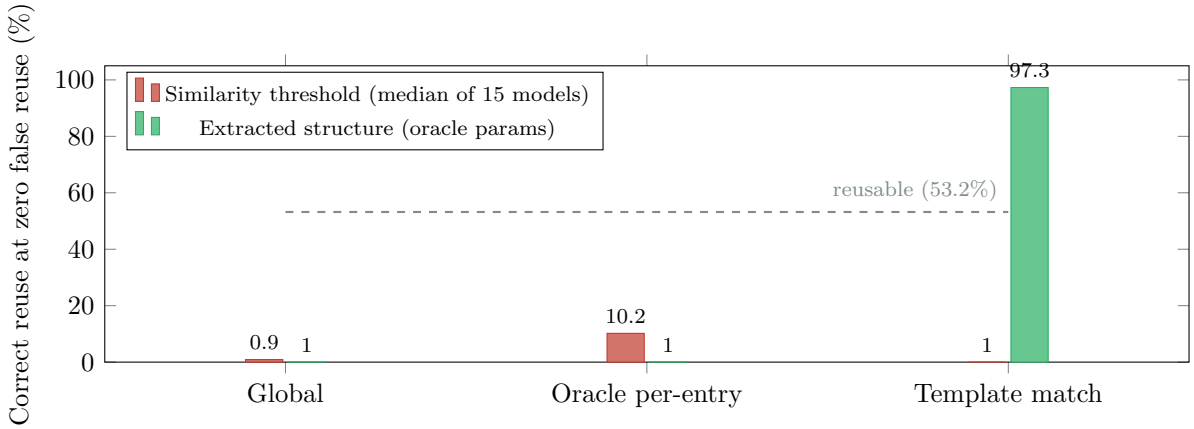


Figure 2: The oracle bound. Even a perfect per-entry threshold recovers only ∼10% of reusable queries (median across 15 models). Template matching with extracted parameters recovers 97%. The dashed line shows the fraction of queries that *should* be reusable. The gap between the oracle bar and the dashed line cannot be closed by any threshold-based method.

3 The Oracle Bound

The central question for threshold-based caching is: if we could pick the *best possible* threshold for every cached entry independently—with full knowledge of every query that will arrive—how much correct reuse could we recover at zero false reuse?

Definition. For each cached entry, find the highest threshold that still produces zero false reuse—the best any system could do with perfect foresight. Set every entry to its own best threshold. This is the oracle per-entry method. Formally, for entry c_i, define $\tau_i^* = \max\{\tau :$ no false reuse of c_i at threshold $\tau\}$. Any method that learns a per-entry threshold [9] is bounded above by this oracle, because the oracle has perfect knowledge of the query distribution.

Results. Table 1 shows the oracle bound across 15 models on the finance dataset (128 cached entries, 53.2% of incoming queries have a reusable match).

Why the bound is so low. The bound is low because the similarity ordering is *inverted*: a one-word period swap ("this year" → "last year") scores higher than any rewording of the same question. In the finance domain, the period swap outscores the best rewording for 78–100% of

Table 2: Exclusion-drop rates by domain (gemini-3.1-flash-lite, pooled across three runs). The included-value column is the same categories in non-negated questions.

Domain	Excluded dropped	Included wrong	Difference [95% CI]
Finance	39/111 (35%)	0/111 (0%)	0.35 [0.21, 0.49]
Retail	39–40/105 (37–38%)	0/104 (0%)	0.37–0.38
Healthcare	5/112 (4%)	0/114 (0%)	0.045 [0.009, 0.080]
IT support	45–46/116 (39–40%)	3/122 (2%)	0.36–0.37

cached entries, in all 15 models (comparison A in the study log). There is no threshold to find because the wrong query is not between the right queries—it is above them.

Implications. This bound applies to any method that makes a reuse decision based solely on a scalar similarity score per entry. It covers global thresholds, per-entry thresholds [9], category-aware thresholds [12], and LLM-as-judge methods that score similarity [10]. The bound does *not* apply to methods that examine the embedding vector's internal structure or that use additional extracted features—those are the structured guards of Section 5.

4 The Exclusion-Drop Asymmetry

During the extraction experiments, we observed an asymmetry not documented in prior error taxonomies [3, 7, 8]: excluded values are dropped far more often than included values.

The finding. When a question says "everything except travel," the extractor should produce `exclusion=travel`. With gemini-3.1-flash-lite, it drops the exclusion in 4–40% of such questions (4% in healthcare, 35–40% in the other three domains). The same category is extracted correctly 100% of the time when the question asks *for* it rather than *excluding* it (Table 2).

Why it matters. A dropped exclusion turns "spending except travel" into "all spending"—a plausible-looking wrong answer with no error signal. The result is that similarity correctly identifies the right template, the guard correctly matches the signature, and the cache still serves wrong SQL because the exclusion was lost during extraction.

Prompt sensitivity. An unrelated change to the extraction prompt's period section doubled the exclusion-drop rate in one experiment (Section 13 of the study log). This suggests the asymmetry is a side effect of how the model distributes attention across the prompt, not a reliable model property.

Caveat. This is primarily a small-model effect. Gemini-3.5-flash (a frontier model at higher cost) dropped zero excluded values in finance, retail and healthcare. The hazard is real for cost-sensitive deployments that use smaller extraction models.

5 Guard Results on Human Questions

The structured guard extracts a signature from the incoming question— operation (sum, count, average), measure (what is being measured), grouping (how results are grouped) and exclusion (what is excluded)—and reuses a cached entry only if the signature matches. This is the design described by Rathore et al. [8] and Bindschaedler [3]. We test it on 480 human-written questions with two extractor vendors.

Table 3: Guard performance on 480 human-written questions, by domain and extractor vendor. Correct reuse: fraction of meaning-preserving questions served from cache. Wrong reuse: fraction served with a wrong template. Ranges span three repeat runs × four embedders.

Domain	Correct reuse (%)		Wrong reuse (%)	
	Gemini	OpenAI	Gemini	OpenAI
Finance	100	98–99	0	0.4
Retail	94–100	85–88	0–5.7	2.1–2.6
Healthcare	90–98	91–100	0.4–7.6	0–8.8
IT support	88–98	77–87	0–11.8	0–13.4
Exact-key baseline: 0–4% correct reuse across all domains				

Results. Table 3 shows correct reuse and wrong reuse rates by domain and vendor. For comparison, an exact-key cache (reuse only when the question string matches exactly) achieves 0–4% correct reuse.

Where the guard leaks. Every wrong reuse on held-out wording was traced to its cause:

- **Retail:** "units sold" was read as a sum in 52 extractions and a count in 59 extractions. When the extractor guesses wrong, the signature matches the wrong template.
- **Healthcare:** "clinic visits" was labeled "inbound" in 83 of 114 extractions. The remaining errors are from templates that share a signature by design (visits ↔ no-shows).
- **IT support:** "top teams" and "top services" have the same extracted signature by design. No extraction error is needed for this false match.

Comparison with prior evaluations. COGNI reports zero false hits on 200 queries constructed by its authors [8]. Bindschaedler reports 82% hits with zero false hits on 1,395 queries including 10 manually authored paraphrases per intent [3]. Our evaluation differs in that the question writers never saw the templates, and the results show domain-specific failure modes that constructed queries may not surface.

6 What the Guard Cannot Catch

Two classes of queries defeat every design tested in this study:

1. **Rolling dates.** "The past year" could mean the rolling 12 months or the calendar year. The extracted period label is the same; the SQL is different. Neither similarity nor the signature distinguishes them.
2. **Explicit date ranges.** "From January 15 to March 1" is a specific interval that no relative label captures. The guard's signature has no slot for arbitrary date ranges.

Relative period labels ("this month," "last quarter") *did* remove every period error in extraction: zero errors in 15,411 extractions across all domains and runs. The fix is effective for the common case but does not cover the long tail of date expressions.

7 Related Work

Semantic caching. GPTCache [1] established the embed-search-reuse pattern. MeanCache [6] argues existing caches give "unacceptable false hit-and-miss rates" and adds context chains.

vCache [9] learns per-entry thresholds; our oracle bound (Section 3) shows this direction has a low ceiling for text-to-SQL. Category-aware thresholds [12] and LLM judges [10] also operate on scalar similarity and fall under the same bound.

Structured guards. COGNI [8] gates reuse on conjunctive checks including entity overlap, intent and polarity. Bindschaedler [3] matches on exact OLAP intent signatures. AWS [4] validates extracted parameter formats. Our study tests this design class on held-out human wording and reports the domain-specific failure modes.

Near-miss queries. TeCoD [7] finds that cosine-threshold template selection performs poorly (63.5% selection accuracy on BIRD) and that "the bulk of the error is due to the wrong literal generation even after matching with the correct template." This is consistent with our finding that template matching works but parameter extraction is fragile.

Embedding blind spots. Negation blindness in text embeddings [5] is the mechanism behind our exclusion-drop finding: the extractor's embedding-based attention gives less weight to negation cues. CacheAttack [13] shows that similarity locality conflicts with collision resistance, achieving 86% hijack rates. The theoretical argument that embeddings can discard the information needed for valid cache hits is made by Zhang et al. [14].

Operational evaluation. Baral et al. [2] argue that "models with the highest PR-AUC are often the worst in operation"—threshold-invariant metrics mask operational failures. Our reporting of correct reuse at fixed false-reuse budgets follows their recommendation.

8 Limitations

Scope. Four analytics domains, one language (English), one question shape (single-table aggregation queries). Multi-table joins, subqueries and non-aggregation queries are not covered.

Extractors. Both extractors are frontier LLMs (Gemini flash-lite, OpenAI gpt-6-luna). Traditional NER or smaller models may behave differently. The exclusion-drop asymmetry was not tested on non-LLM extractors.

No production traffic. All questions are either template-generated (synthetic) or volunteer-written (human). Neither represents the distribution of a production analytics tool. The guard's performance on production traffic is an open question.

Model breadth vs. depth. We tested 15 models but all on the same task. Four or five models spanning open and commercial publishers would have supported the same conclusions. The additional 10 models confirm universality but the marginal value of each additional model is small.

Pre-registration. Pre-registration is evidence of discipline, not proof of correctness. It prevents post-hoc threshold tuning and selective reporting, but the pre-registered decision rules themselves could be poorly chosen.

9 Conclusion

The threshold-tuning direction for text-to-SQL semantic caching has a hard ceiling. An oracle per-entry threshold recovers at most 6–17% of reusable queries at zero false reuse, across 15 embedding models spanning five years. The information needed to distinguish safe from unsafe reuse is not in the similarity score.

Structured guards that match on extracted signatures—a design proposed by Rathore et al. [8] and Bindschaedler [3]—achieve 77–100% correct reuse on human-written questions. The

guard leaks are domain-specific and traceable to extraction ambiguity, not to the guard design itself.

The residual risk is date semantics. Rolling periods and explicit date ranges produce wrong SQL that no design tested here catches. This is the open problem.

Reproducibility. The dataset (480 human-written questions, four domains), all model outputs, extraction results and evaluation code are available at https://github.com/rbobba/semantic-cache-inversion. The `verify_claims.py` script recomputes every number cited in the study log from the files on disk.

References

[1] Fu Bang. GPTCache: An open-source semantic cache for LLM applications. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS), ACL*, pages 212–218, 2023. doi: 10.18653/v1/2023.nlposs-1.24.

[2] Aditeya Baral et al. The operational gap in semantic cache evaluation. In *EMNLP Industry Track*, 2026. arXiv:2606.19719.

[3] Laurent Bindschaedler. Semantic caching for OLAP via LLM-based query canonicalization. *arXiv preprint arXiv:2602.19811*, 2026. Extended version of a short paper at DOLAP 2026 (with EDBT/ICDT).

[4] Brukau, Rudolph, and Bhat. Reducing Text2SQL latency with parameterized query templates. AWS Architecture Blog, August 2026.

[5] Hongliu Cao. Semantic adapter for universal text embeddings: Diagnosing and mitigating negation blindness to enhance universality. In *ECAI*, 2025. arXiv:2504.00584.

[6] Waris Gill et al. MeanCache: User-centric semantic caching for large language model based web services. In *IPDPS*, 2025. arXiv:2403.02694.

[7] Smit Jivani, Sarvam Maheshwari, and Sunita Sarawagi. Reliable answers for recurring questions: Boosting Text-to-SQL accuracy with template constrained decoding. *Proceedings of the ACM on Management of Data (PACMMOD)*, 3(6), 2026. arXiv:2604.28028.

[8] Darshita Rathore, Vineet Kumar, Vaibhav Singal, Ankur Vivek Singh, and Anindya Moitra. Conversational query engine for mixed-modality heterogeneous enterprise data sources. *arXiv preprint arXiv:2606.28370*, 2026. Agent4IR Workshop, KDD 2026.

[9] Luis Gaspar Schroeder et al. vCache: Verified semantic prompt caching. In *ICLR*, 2026. arXiv:2502.03771.

[10] Asmit Kumar Singh et al. Asynchronous verified semantic caching for tiered LLM architectures. *arXiv preprint arXiv:2602.13165*, 2026.

[11] Harmohit Singh. Semantic caching and intent-driven context optimization for multi-agent natural language to code systems. *arXiv preprint arXiv:2601.11687*, 2026.

[12] Chen Wang et al. Category-aware semantic caching for heterogeneous LLM workloads. *arXiv preprint arXiv:2510.26835*, 2025. Position paper.

[13] Zhixiang Zhang, Zesen Liu, Yuchong Xie, Quanfeng Huang, and Dongdong She. From similarity to vulnerability: Key collision attack on LLM semantic caching. In *ICML*, 2026. arXiv:2601.23088.

[14] Zihan Zhang et al. Similarity is not validity: Defending LLM semantic caches against poisoning. *arXiv preprint arXiv:2609.35908*, 2026.

A Full Oracle Bound Table

Table 4 shows the oracle bound for all 15 models on the finance dataset.

Table 4: Correct reuse at zero false reuse (%), all 15 models.

Model (year)	Global thresh.	Oracle per-entry	Template (oracle)	split_full (oracle)
all-MiniLM-L6-v2 (2021)	0.8	7.1	97.3	21.3
all-mpnet-base-v2 (2021)	1.8	6.6	96.2	21.5
e5-base-v2 (2023)	0.9	8.1	97.3	21.4
bge-base-en-v1.5 (2023)	0.5	8.1	97.3	21.8
text-embedding-3-small (2024)	0.2	6.4	97.3	21.4
text-embedding-3-large (2024)	1.2	9.8	97.3	22.5
mxbai-embed-large-v1 (2024)	0.6	10.6	100.0	22.6
snowflake-arctic-embed (2024)	2.1	10.7	97.3	22.4
text-embedding-004 (2024)	0.5	9.3	97.3	24.2
text-emb-004 SemSim (2024)	1.1	11.0	97.3	27.3
voyage-3-large (2025)	1.1	14.4	100.0	27.2
voyage-3.5 (2025)	0.6	11.4	100.0	24.9
gemini-embedding-001 (2025)	0.2	10.2	100.0	32.0
Qwen3-Embedding-0.6B (2025)	1.1	11.2	97.3	21.5
voyage-4 (2026)	6.3	16.8	97.3	30.0
Range	*0.2–6.3*	*6.4–16.8*	*96.2–100.0*	*21.3–32.0*

B Comparison A Across Domains

Comparison A measures: for each cached entry, does the period swap score higher than the best rewording? Table 5 shows the rate across four domains for representative models.

Table 5: Comparison A: fraction of cached entries where the period swap outscores the best rewording (higher = worse for caching). Four representative models shown; all 15 behave similarly.

Model	Finance	Retail	Healthcare	IT support
bge-base-en-v1.5	0.875	0.922	0.945	0.960
text-emb-3-large	0.945	0.984	0.972	0.992
gemini-emb-001	0.898	0.930	0.931	0.984
voyage-4	0.789	0.836	0.793	0.912

C AUC on Human Questions

Table 6 shows the AUC for separating rewordings from period changes on 480 human-written questions. All 15 models cluster near the coin-toss line (0.50).

Table 6: AUC for rewording vs. period-change separation on human questions. An AUC of 0.50 is chance. The pre-registered decision rule required all upper confidence bounds below 0.80; this holds.

Model	AUC	95% CI
all-MiniLM-L6-v2	0.526	[0.449, 0.608]
all-mpnet-base-v2	0.522	[0.443, 0.604]
bge-base-en-v1.5	0.580	[0.507, 0.655]
e5-base-v2	0.581	[0.506, 0.662]
Qwen3-Embedding-0.6B	0.595	[0.519, 0.677]
Snowflake Arctic	0.561	[0.486, 0.639]
mxbai-embed-large	0.545	[0.474, 0.619]
text-embedding-004	0.524	[0.449, 0.602]
text-emb-004 SemSim	0.559	[0.480, 0.639]
gemini-embedding-001	0.559	[0.480, 0.634]
text-embedding-3-small	0.581	[0.505, 0.665]
text-embedding-3-large	0.594	[0.513, 0.681]
voyage-3-large	0.595	[0.520, 0.672]
voyage-3.5	0.550	[0.475, 0.628]
voyage-4	0.595	[0.528, 0.671]